
Beyond Freeze-or-Fine-Tune: Structured Transfer Learning under Domain Shift

Sofia Lambro
Bocconi University
sofia.lambro@studbocconi.it

Alessandro Lombardi
Bocconi University
alessandro.lombardi2@studbocconi.it

Marco Riva
Bocconi University
marco.riva4@studbocconi.it

Katia Russo
Bocconi University
katia.russo@studbocconi.it

Tommaso Tarantino
Bocconi University
tommaso.tarantino@studbocconi.it

Abstract

Modern transfer learning offers many adaptation strategies, from frozen-feature baselines to full and parameter-efficient fine-tuning. We revisit this design space through a simple, interpretable question: which depths of a convolutional backbone should be reused, adapted, or relearned? We study monotonic Frozen/Fine-tuned/Scratch ($F/T/S$) partitions, where contiguous backbone regions are frozen, fine-tuned, or trained from scratch. First, we replicate and extend the layer-transfer analysis of Yosinski et al. (2014) on TinyImageNet with WRN-28-10 trained from scratch on disjoint class splits. This controlled setting shows that early and intermediate features transfer reliably, while fully frozen deep cross-domain features collapse; it also reveals an optimization instability when a large scratch block is placed immediately after a fine-tuned block. Second, we test the same design principle on ImageNet-pretrained ResNet50 under real distribution shift using iWildCam from WILDS. Sweeping all valid $F/T/S$ partitions shows that transfer performance is strongly configuration-dependent: fully frozen backbones underperform, substantial fine-tuning is needed under domain shift, and mixed partitions provide useful performance–efficiency trade-offs. Overall, our results show that depth-wise allocation of reuse, adaptation, and relearning is a useful diagnostic lens for understanding transfer behavior under both controlled and real-world domain shift.

1 Introduction

Transfer learning is a standard tool in computer vision: a backbone trained on a large source dataset is reused on a target task to improve accuracy, reduce data requirements, or lower training cost. Contemporary adaptation practice is broader than a simple freeze-versus-fine-tune choice, including partial fine-tuning and parameter-efficient methods. Still, frozen-feature transfer and full fine-tuning remain central reference points, and the depth-wise structure of what should be reused, adapted, or relearned is often less explicit. Since convolutional networks build hierarchical representations, from low-level visual primitives to increasingly task-specific features, different depths may require different adaptation decisions.

We study this question through a deliberately simple and interpretable design. Given a backbone divided into contiguous blocks, each block is assigned one of three roles: frozen (F), fine-tuned (T), or trained from scratch (S). We restrict attention to monotonic configurations $F^a T^b S^c$, where frozen blocks form a lower feature prefix, fine-tuned blocks provide an adaptation region, and scratch blocks are closest to the task-specific head. This extends the classical layer-transfer setup of Yosinski et al. (2014) from binary transferred prefixes to a ternary reuse–adapt–relearn partition.

Our study has two parts. First, following the selfer/transfer distinction of Yosinski et al. (2014), we run a controlled TinyImageNet experiment with WRN-28-10 trained from scratch on disjoint class splits, comparing same-domain runs where source and target class subsets match with cross-domain runs where they differ. This separates two classical sources of transfer degradation: co-adaptation fragility, visible even when source and target tasks match, and feature specialization, visible when deep features are copied across disjoint class subsets. Second, we test the same $F/T/S$ principle in a practical setting: ImageNet-pretrained ResNet50 adapted to iWildCam, a WILDS benchmark with real location shift and class imbalance (Koh et al., 2021). Across both settings, the goal is not to propose a new state-of-the-art adaptation method, but to map how transfer behavior changes when representation depth is allocated to reuse, adaptation, or relearning.

2 Background and Related Work

Deep visual features trained on large source datasets can often be reused on downstream tasks, as shown by early generic-feature transfer work such as DeCAF (Donahue et al., 2014). However, transferability is depth-dependent: lower layers tend to encode generic primitives, whereas deeper layers become more specialized to the source task (Yosinski et al., 2014; Kornblith et al., 2019). The closest reference point for our controlled experiment is Yosinski et al. (2014), which copied the first n layers of a source network into a target network, either freezing or fine-tuning them, and showed both feature-specificity and co-adaptation effects. Our TinyImageNet experiment preserves this selfer-versus-transfer logic, but extends the binary frozen/fine-tuned prefix design to a ternary monotonic partition $F^a T^b S^c$.

Modern adaptation research also includes parameter-efficient fine-tuning methods, such as adapters, low-rank updates, and prompt-based tuning, especially for large vision and vision-language models (Hu et al., 2022; Jia et al., 2022; Xin et al., 2024). Our aim is different and more diagnostic: we keep the backbone architecture fixed and exhaustively map which contiguous depth regions should be reused, adapted, or relearned. We test this design on iWildCam, a WILDS benchmark where OOD (Out Of Domain) examples come from unseen camera-trap locations (Koh et al., 2021).

3 Method

Let a convolutional backbone be decomposed into L ordered functional blocks $B_1, \dots, B_L$, from early feature extractors to deeper task-specific stages. Here L denotes the number of backbone blocks, not the number of individual layers. Each block receives a state

$$z_\ell \in \{F, T, S\}, \quad \ell = 1, \dots, L,$$

where F denotes copied and frozen weights, T copied and trainable weights, and S randomly reinitialized trainable weights. The classifier head is always trained from scratch because the target label space differs from the source label space.

We restrict the search to monotonic configurations

$$F^a T^b S^c, \quad a + b + c = L,$$

so that

$$z_\ell = \begin{cases} F, & 1 \leq \ell \leq a, \\ T, & a < \ell \leq a + b, \\ S, & a + b < \ell \leq L. \end{cases}$$

This gives $(L+1)(L+2)/2$ valid block-level configurations: 15 for the WRN-28-10 study with four backbone groups, and 21 for the ResNet50 study with five backbone stages.

The monotonic constraint is a control for interpretability and feature compatibility. Frozen prefixes test how much source representation can be reused; fine-tuned regions test how much adaptation is needed; scratch suffixes test whether high-level representations should be relearned. We exclude non-monotonic assignments because a reinitialized block placed below copied blocks would feed them feature distributions they were not trained to process. Similarly, interleaved patterns would introduce several representation shifts along depth, making it difficult to distinguish transfer failure from architecture-induced mismatch. Placing S only at the top confines relearning to task-specific stages, while T can act as a transition region between fixed source features and newly learned target features.

For target data $\mathcal{D}_t = \{(x_i, y_i)\}_{i=1}^n$, each configuration defines

$$f_{\theta}^{(a,b,c)} = h_{\phi} \circ B_L^{z_L} \circ \dots \circ B_1^{z_1},$$

with trainable parameters

$$\Theta_{\text{train}}^{(a,b,c)} = \{\theta_{\ell} : z_{\ell} \in \{T, S\}\} \cup \{\phi\}.$$

Training minimizes the target supervised loss over $\Theta_{\text{train}}^{(a,b,c)}$, while all F parameters are held fixed.

4 Experiments

We evaluate the $F/T/S$ formulation in two regimes: a controlled TinyImageNet layer-transfer study and a realistic ImageNet-to-iWildCam transfer study.

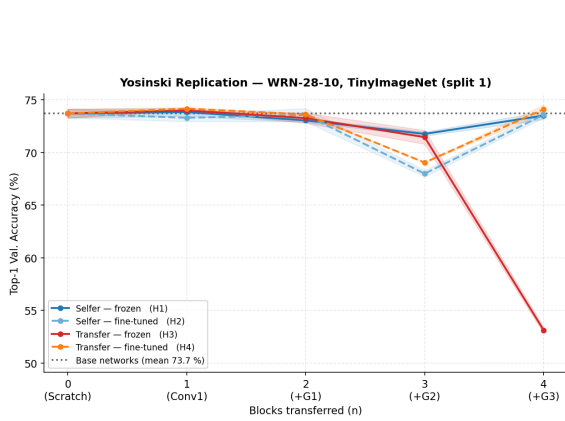
Controlled TinyImageNet transfer. We split TinyImageNet into two disjoint 100-class subsets, A and B , and train two WRN-28-10 source networks from scratch, baseA and baseB. The backbone is divided into four blocks: Conv1, Group1, Group2, and Group3. With $L = 4$, there are 15 conceptual $F/T/S$ configurations: the source-free all-scratch case is represented by baseA/baseB, while the other 14 configurations are trained for the four selfer/transfer directions $A \rightarrow A$, $B \rightarrow B$, $A \rightarrow B$, and $B \rightarrow A$, giving $2 + 14 \cdot 4 = 58$ models. We report top-1 accuracy and TinyImageNet-C-style corruption accuracy (Hendrycks and Dietterich, 2019); optimization details are in Appendix A.

Real-world iWildCam transfer. We adapt ImageNet-pretrained ResNet50 (He et al., 2016) to iWildCam, where OOD examples come from unseen camera-trap locations (Koh et al., 2021). ResNet50 is divided into five backbone blocks: Stem, Stage1, Stage2, Stage3, and Stage4, yielding 21 valid backbone partitions. Unlike the TinyImageNet labels, which omit the always-scratch head, the iWildCam experiment logs include the head explicitly as a final $S1$ (e.g. T5S1 denotes five fine-tuned backbone blocks plus a scratch head, while S6 denotes five scratch backbone blocks plus a scratch head). Since iWildCam is class-imbalanced, we emphasize macro-level metrics: OOD balanced accuracy (macro recall) for partition analysis and OOD macro-F1 for WILDS/iWildCam comparability; raw accuracy and trainable-parameter fraction are reported as secondary quantities. Hyperparameters and time-budget details are reported in Appendix A.

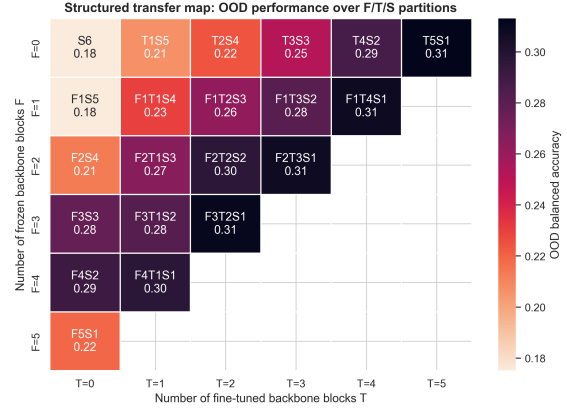
4.1 Results

Controlled transfer exposes depth-specific failure modes. Figure 1a shows that TinyImageNet qualitatively reproduces the layer-transfer phenomena of Yosinski et al. (2014), but with architecture-dependent magnitudes. In the selfer frozen curve, co-adaptation fragility is present but small: accuracy drops from 73.7% to 71.8% when Conv1–Group2 are frozen, then recovers to 73.5% when all four blocks are frozen. This attenuation is consistent with WRN-28-10 residual connections reducing the brittleness of frozen intermediate representations. In cross-domain frozen transfer, feature specialization is much stronger: performance remains near baseline through Conv1–Group2, but FFFF collapses to 53.1%, versus 73.5% for selfer FFFF. Thus, the deepest residual group is reusable as a fixed representation only when source and target class distributions match.

The ternary $F/T/S$ sweep reveals a failure mode invisible to binary frozen/scratch experiments. Whenever Group2 is fine-tuned while Group3 is scratch (TTTS, FTTS, and FFTS) accuracy collapses to roughly 67–69%, independently of the frozen prefix and of whether the run is selfer or transfer. Since Group3 contains most WRN parameters, this is consistent with a moving-target optimization problem: a large scratch block is trained at high learning rate while its input distribution drifts



(a) TinyImageNet depth curves.



(b) iWildCam OOD balanced accuracy.

Figure 1: **Controlled and real-world transfer landscapes.** Left: Yosinski-style WRN-28-10 transfer on TinyImageNet. Right: ResNet50 $F/T/S$ sweep on iWildCam, with scratch depth $S = 5 - F - T$.

because the upstream block is being fine-tuned. Moving Group3 out of scratch restores near-baseline performance.

TinyImageNet-C does not reveal a separate clean-robust trade-off: corruption accuracy closely tracks clean accuracy across the 56 source-dependent runs ($r = 0.980$).

Real-world transfer pushes the optimum toward adaptation. The iWildCam sweep shows the same depth-dependence under stronger real-world shift. The fully frozen baseline F5S1 is worse than scratch on raw accuracy (ID/OOD 42.8/22.8% versus 56.5/30.3% for S6), showing that a fixed ImageNet backbone plus a new head is too rigid. Conversely, full fine-tuning T5S1 gives the best OOD balanced accuracy (31.3%), raw OOD accuracy (55.3%), and OOD macro-F1 (29.5%). The decisive factor is the last residual stage: replacing scratch Stage4 with fine-tuned Stage4 gives the largest OOD gain in the sweep, with T5S1 improving over T4S2 by +11.2 pp in raw OOD accuracy.

The efficiency trade-off is also concentrated near Stage4. Fine-tuning only Stage4 (F4T1S1) reaches 49.3% raw OOD accuracy at about 76% of full fine-tuning FLOPs, while F3T2S1 and F2T3S1 approach full-finetuning performance at near-full compute. Thus, mixed partitions are not universally superior to full fine-tuning. They show that early generic layers can often be frozen with little cost, whereas high-capacity semantic stages must be adapted.

Across the 21 configurations, ID and OOD metrics are strongly aligned, so the sweep does not expose an ID-OOD trade-off: weak partitions fail both in-domain and under location shift.

Further results and analysis are present in Appendix C and D.

5 Conclusion

We studied transfer learning as a depth-wise allocation problem using monotonic frozen, fine-tuned, and scratch ($F/T/S$) backbone partitions. In the controlled TinyImageNet experiment, early and intermediate features transferred reliably, while fully frozen deep cross-domain features collapsed; the ternary sweep also revealed a scratch-after-fine-tune instability at the Group2→Group3 boundary. In the iWildCam experiment, real distribution shift required substantial adaptation: freezing the full ImageNet backbone was worse than scratch, while full fine-tuning achieved the best OOD performance.

Overall, the results show that transfer behavior depends on how reuse, adaptation, and relearning are allocated across depth. Mixed partitions are not a universal replacement for full fine-tuning, but they expose where adaptation is necessary and where freezing remains safe. The main limitations are the lack of multiple seeds/splits, the use of relatively simple CNN-based settings, and the absence

of more advanced adaptation methods such as LoRA-style or other parameter-efficient fine-tuning techniques. Future work should test higher-resolution or more challenging datasets, finer architectural decompositions, and modern pretrained backbones such as Vision Transformers or self-supervised models (Appendix E).

References

- Jeff Donahue, Yangqing Jia, Oriol Vinyals, Judy Hoffman, Ning Zhang, Eric Tzeng, and Trevor Darrell. DeCAF: A deep convolutional activation feature for generic visual recognition. In *International Conference on Machine Learning*, 2014.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2016.
- Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. In *International Conference on Learning Representations*, 2019.
- Edward J. Hu, Yelong Shen, Phil Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Menglin Jia, Luming Tang, Bor-Chun Chen, Claire Cardie, Serge Belongie, Bharath Hariharan, and Ser-Nam Lim. Visual prompt tuning. In *European Conference on Computer Vision*, 2022.
- Pang Wei Koh, Shiori Sagawa, Henrik Marklund, Sang Michael Xie, Marvin Zhang, Akshay Bal-subramani, Weihua Hu, Michihiro Yasunaga, Richard Lanus Phillips, Irena Gao, et al. WILDS: A benchmark of in-the-wild distribution shifts. In *International Conference on Machine Learning*, 2021.
- Simon Kornblith, Jonathon Shlens, and Quoc V. Le. Do better ImageNet models transfer better? In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019.
- Behnam Neyshabur, Hanie Sedghi, and Chiyuan Zhang. What is being transferred in transfer learning? In *Advances in Neural Information Processing Systems*, 2020.
- Yi Xin, Siqi Luo, Haodi Zhou, Junlong Du, Xiaohong Liu, Yue Fan, Tao Du, et al. Parameter-efficient fine-tuning for pre-trained vision models: A survey. *arXiv preprint arXiv:2402.02242*, 2024.
- Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems*, 2014.

A Implementation Details

A.1 Experiment 1: TinyImageNet

Dataset and class split. TinyImageNet is a 200-class, ImageNet-derived image classification dataset with images resized to 64×64 . Each class contains 500 training images and 50 validation images. We split the 200 classes into two disjoint 100-class subsets, denoted A and B , so that source and target tasks never share label identities.

The split is not a purely uniform random split over the 200 classes. Classes are first assigned to six semantic groups by matching TinyImageNet WordNet labels: mammals, other fauna, human spaces, vehicles, food, and objects/tools/misc. Within each semantic group, classes are shuffled using the split seed and divided between A and B , preserving group balance as closely as possible. The resulting split has exactly 100 classes in A and 100 in B . For the two odd-sized groups, split 1 assigns 16/17 other-fauna classes and 9/8 food classes to A/B , respectively; all even-sized groups are split exactly in half. This makes the two subsets disjoint but semantically comparable.

Model and block decomposition. We use WRN-28-10 as the backbone for the controlled transfer experiment. The model follows the standard WideResNet structure with an initial convolution followed by three residual groups. For a 64×64 TinyImageNet input, the four controlled backbone blocks are:

Conv1, Group1, Group2, Group3.

Conv1 maps RGB inputs to 16 channels at 64×64 . Group1 keeps spatial resolution and widens the representation to 160 channels. Group2 downsamples to 32×32 and outputs 320 channels. Group3 downsamples to 16×16 and outputs 640 channels. The classification head consists of the final normalization/ReLU, global average pooling, and a linear classifier. The head is always initialized from scratch and trained on the target label space.

Table 1: **WRN-28-10 block size on TinyImageNet.** Parameter and forward-FLOP contributions are reported per controlled block. Percentages are relative to the full model, including the classification head.

Block	Parameters	%	Forward FLOPs	%
Conv1	432	(0.001%)	0.0018G	(0.007%)
Group1	1,640,672	(4.49%)	6.7202G	(28.21%)
Group2	6,968,000	(19.07%)	8.5538G	(35.90%)
Group3	27,862,400	(76.26%)	8.5498G	(35.88%)
Classification head	65,380	(0.18%)	0.0006G	(0.002%)
Total	36,536,884	(100.00%)	23.8261G	(100.00%)

Transfer protocol. We train two WRN-28-10 base models from scratch: baseA on subset A and baseB on subset B . These two base models instantiate the source-independent all-scratch condition. For every source-dependent $F/T/S$ configuration, we train four source–target directions:

$$A \rightarrow A, \quad B \rightarrow B, \quad A \rightarrow B, \quad B \rightarrow A.$$

The same-domain directions $A \rightarrow A$ and $B \rightarrow B$ are selfer controls in the sense of Yosinski et al. (2014): source and target label spaces match, so degradation mainly reflects co-adaptation or optimization effects. The cross-domain directions $A \rightarrow B$ and $B \rightarrow A$ test transfer across disjoint class subsets and therefore additionally expose feature specialization.

All WRN-28-10 models are trained for 100 epochs using SGD with momentum 0.9, weight decay $5 \cdot 10^{-4}$, cosine learning-rate scheduling, and batch size 128. Scratch backbone blocks and the classifier head use learning rate 0.1, while fine-tuned transferred blocks use learning rate 0.01. Frozen blocks are excluded from optimization. Batch-normalization layers whose affine parameters are frozen are kept in evaluation mode, preserving source-domain running statistics in frozen blocks.

TinyImageNet-C-style corruption evaluation. To evaluate synthetic robustness in Experiment 1, we construct a TinyImageNet-C-style corrupted validation set using implementations matching the ImageNet-C corruption functions from Hendrycks and Dietterich (2019). The code also supports frost overlays from the public `hendrycks/robustness` repository, although the selected corruption set used in this experiment is:

`{brightness, contrast, defocus blur, fog, saturate, snow}`.

For each target subset, we build a corrupted validation set of 5000 images. Validation images are shuffled with a fixed seed and partitioned into six non-overlapping groups, one per corruption. Each corruption has a maximum severity, and the actual severity for each image is sampled independently and uniformly from $\{1, \dots, s_{\max}\}$. The severity ranges are:

Corruption	Severity range
brightness	$\{1, 2, 3, 4\}$
contrast	$\{1, 2, 3\}$
defocus blur	$\{1, 2\}$
fog	$\{1, 2, 3, 4, 5\}$
saturate	$\{1, 2, 3\}$
snow	$\{1, 2, 3, 4\}$

The corrupted filename stores both the corruption type and sampled severity, and the manifest records the original filename, class, corruption, severity, and maximum severity. We report corruption accuracy as top-1 accuracy averaged over corrupted validation examples. This is used as a synthetic stress test and is not treated as a substitute for real distribution-shift evaluation.

A.2 Experiment 2: iWildCam

Dataset. For the real-world transfer experiment, we use iWildCam from WILDS. iWildCam is a camera-trap wildlife classification benchmark designed to evaluate generalization across naturally occurring distribution shifts, with OOD examples coming from unseen camera-trap locations. This produces changes in background, illumination, geography, camera viewpoint, class priors, and animal appearance. The source model is pretrained on ImageNet-1K, while the target task is iWildCam classification.

Model and block decomposition. We use a torchvision ResNet50 initialized with ImageNet-1K pretrained weights when at least one backbone block is in state F or T . For the all-scratch configuration S6, no ImageNet weights are loaded. The ResNet50 backbone is decomposed into five controlled blocks:

Stem, Stage1, Stage2, Stage3, Stage4.

The stem contains the initial convolution and batch normalization. Stage1–Stage4 correspond to the standard ResNet residual layers `layer1`, `layer2`, `layer3`, and `layer4`. The classification head is the final fully connected layer, optionally preceded by dropout; it is always initialized from scratch to match the iWildCam label space. Frozen blocks keep ImageNet-pretrained weights fixed, fine-tuned blocks start from ImageNet-pretrained weights and are optimized, and scratch blocks are reinitialized with Kaiming/Xavier-style initialization and optimized.

Table 2: **ResNet50 block size on iWildCam.** Parameter and forward-FLOP contributions are reported per controlled block. Percentages are relative to the full model, including the scratch iWildCam classification head.

Block	Parameters	%	Forward FLOPs	%
Stem	9,536	(0.04%)	0.1196G	(2.91%)
Stage1	215,808	(0.90%)	0.6768G	(16.47%)
Stage2	1,219,584	(5.11%)	1.0338G	(25.15%)
Stage3	7,098,368	(29.72%)	1.4687G	(35.74%)
Stage4	14,964,736	(62.66%)	0.8105G	(19.72%)
Classification head	372,918	(1.56%)	0.0005G	(0.012%)
Total	23,880,950	(100.00%)	4.1098G	(100.00%)

Training protocol. All iWildCam configurations use images resized/cropped to 224×224 with ImageNet normalization. Training augmentation consists of random resized crop, horizontal flip, and color jitter; validation uses resize and center crop. We train with AdamW, cosine learning-rate scheduling, batch size 32, backbone learning rate 10^{-4} , head learning rate 10^{-3} , weight decay 10^{-3} , label smoothing 0.1, head dropout 0.3, and inverse-frequency weighted sampling to mitigate class imbalance. Runs follow a time-budgeted protocol and typically complete approximately 40 epochs.

Evaluation metrics. For iWildCam, raw accuracy is not sufficient because the dataset is long-tailed. We therefore report raw accuracy, balanced accuracy, and macro-F1 on both ID and OOD validation splits. Balanced accuracy is equivalent to macro recall in the multiclass setting and is used as our main diagnostic metric for comparing $F/T/S$ partitions. Macro-F1 is reported alongside it because it is the standard WILDS/iWildCam-compatible macro metric and is especially relevant for rare classes. Unless otherwise stated, model rankings in the main text use OOD balanced accuracy, while OOD macro-F1 is used as a secondary robustness check.

B Configuration Spaces

B.1 TinyImageNet WRN-28-10 Configurations

With four backbone blocks, the monotonic configuration space contains

$$\frac{(4+1)(4+2)}{2} = 15$$

valid partitions. The configurations are:

SSSS,
 FSSS, TSSS,
 FFSS, FTSS, TTSS,
 FFFS, FFTS, FTTS, TTTS,
 FFFF, FFFT, FFTT, FT TT, TTTT.

Equivalently, each configuration is of the form $F^a T^b S^c$ with $a + b + c = 4$. The head is not included in this notation because it is always scratch.

B.2 ResNet50 iWildCam Configurations

With five backbone blocks, the monotonic backbone configuration space contains

$$\frac{(5+1)(5+2)}{2} = 21$$

valid partitions of the form $F^a T^b S^c$ with $a + b + c = 5$. In the main methodological notation, the classifier head is not counted in L and is always scratch. However, iWildCam labels explicitly append the scratch head as a final $S1$. The final $S1$ should therefore not be interpreted as an additional backbone block.

The 21 logged iWildCam configurations are:

S6,
F1S5, T1S5,
F2S4, F1T1S4, T2S4,
F3S3, F2T1S3, F1T2S3, T3S3,
F4S2, F3T1S2, F2T2S2, F1T3S2, T4S2,
F5S1, F4T1S1, F3T2S1, F2T3S1, F1T4S1, T5S1.

C Full Results

C.1 iWildCam Board

Table 3 reports all 21 ResNet50 $F/T/S$ configurations, ranked by OOD balanced accuracy. Balanced accuracy is the primary metric because iWildCam is class-imbalanced; raw accuracy and macro-F1 are included for completeness.

Table 3: **Full iWildCam leaderboard over all 21 $F/T/S$ configurations.** Configurations are ranked by OOD balanced accuracy. Train. is the percentage of trainable parameters.

Config	Train.	ID Acc.	ID Bal.	ID F1	OOD Acc.	OOD Bal.	OOD F1
T5S1	100.0	73.5	41.8	35.5	55.3	31.3	29.5
F3T2S1	93.9	71.2	41.9	35.9	50.8	31.1	26.8
F1T4S1	100.0	70.8	42.5	36.0	51.7	30.9	26.3
F2T3S1	99.1	73.2	42.7	36.2	51.7	30.8	26.9
F4T1S1	64.2	69.5	41.4	34.5	49.3	29.8	23.5
F2T2S2	99.1	68.9	41.2	34.6	48.8	29.6	25.0
F4S2	64.2	67.1	40.1	33.7	47.1	29.1	23.9
T4S2	100.0	68.7	41.4	34.8	44.1	28.8	24.6
F3T1S2	93.9	67.7	41.0	33.2	47.0	28.2	23.1
F1T3S2	100.0	67.9	40.4	32.4	47.5	28.2	22.7
F3S3	93.9	64.1	38.4	31.7	41.2	27.7	21.4
F2T1S3	99.1	66.1	38.1	31.0	41.9	26.6	21.3
F1T2S3	100.0	64.2	37.3	30.1	44.2	26.3	20.7
T3S3	100.0	65.8	37.4	29.9	40.4	25.2	18.8
F1T1S4	100.0	64.3	33.9	28.0	40.8	23.0	19.6
T2S4	100.0	62.8	35.1	28.4	39.6	22.5	18.1
F5S1	1.6	42.8	35.3	26.3	22.8	21.9	15.2
F2S4	99.1	59.0	34.1	28.0	39.8	21.3	16.7
T1S5	100.0	58.4	34.2	26.7	33.3	20.7	14.8
F1S5	100.0	57.5	32.3	25.9	33.0	17.6	15.0
S6	100.0	56.5	30.7	24.1	30.3	17.5	14.1

C.2 TinyImageNet Board

Table 4: **TinyImageNet selfer configurations.** Same-domain results averaged over $A \rightarrow A$ and $B \rightarrow B$. Clean accuracy uses best validation top-1; corrupt accuracy uses TinyImageNet-C-style top-1. Δ is relative to the source-free SSSS baseline.

Family	Config	Clean (ID)	Δ Clean	Corrupt (OOD)	Δ Corrupt
F/S	SSSS	73.7	+0.0	58.9	+0.0
F/S	FSSS	73.8	+0.1	59.7	+0.8
F/S	FFSS	73.0	-0.7	57.9	-1.0
F/S	FFFS	71.8	-1.9	56.4	-2.5
F/S	FFFF	73.5	-0.2	58.9	+0.0
T/S	TSSS	73.3	-0.4	59.3	+0.4
T/S	TTSS	73.6	-0.1	58.3	-0.6
T/S	TTTS	68.0	-5.7	53.3	-5.6
T/S	TTTT	73.6	-0.1	59.3	+0.4
F/T/S	FTSS	73.2	-0.5	58.1	-0.8
F/T/S	FFTS	66.9	-6.8	52.2	-6.7
F/T/S	FTTS	68.2	-5.5	53.3	-5.6
F/T/S	FFFT	73.9	+0.2	59.0	+0.1
F/T/S	FFTT	73.6	-0.1	59.0	+0.1
F/T/S	FTTT	73.3	-0.4	59.7	+0.8

Table 5: **TinyImageNet transfer configurations.** Cross-domain results averaged over $A \rightarrow B$ and $B \rightarrow A$. Clean accuracy uses best validation top-1; corrupt accuracy uses TinyImageNet-C-style top-1. Δ is relative to the source-free SSSS baseline.

Family	Config	Clean (ID)	Δ Clean	Corrupt (OOD)	Δ Corrupt
F/S	SSSS	73.7	+0.0	58.9	+0.0
F/S	FSSS	74.0	+0.3	60.1	+1.2
F/S	FFSS	73.3	-0.4	57.7	-1.2
F/S	FFFS	71.4	-2.3	56.6	-2.2
F/S	FFFF	53.1	-20.6	38.9	-20.0
T/S	TSSS	74.2	+0.5	59.4	+0.5
T/S	TTSS	73.6	-0.1	58.6	-0.3
T/S	TTTS	69.0	-4.7	54.1	-4.8
T/S	TTTT	74.1	+0.4	59.6	+0.7
F/T/S	FTSS	73.1	-0.6	57.8	-1.1
F/T/S	FFTS	67.8	-5.9	52.9	-6.0
F/T/S	FTTS	69.2	-4.5	54.0	-4.9
F/T/S	FFFT	71.9	-1.8	57.4	-1.5
F/T/S	FFTT	73.8	+0.1	59.0	+0.1
F/T/S	FTTT	74.2	+0.5	60.1	+1.2

D Extended Results and Discussion

This appendix expands the main empirical findings. We group the results by experiment and focus on the observations that support the paper’s central claim: transfer behavior depends on how reuse, adaptation, and relearning are allocated across depth.

D.1 Experiment 1: Controlled TinyImageNet Transfer

Experiment 1 is a diagnostic replication and extension of the layer-transfer setting of Yosinski et al. (2014). Its goal is to separate two mechanisms that are usually entangled in transfer learning: co-adaptation fragility, which can appear even when source and target tasks match, and feature specialization, which appears when deep features are copied across different label spaces.

Split validity. Before interpreting the transfer curves, we checked that the two TinyImageNet class halves were comparable. Matched conditions across the two directions are highly correlated: $r = 0.996$ for transfer and $r = 0.971$ for selfer. Thus, the main effects are unlikely to be artifacts of one class subset being systematically easier than the other. The class split is still a limitation because it is only one split, but the symmetry check supports the internal validity of this controlled setting.

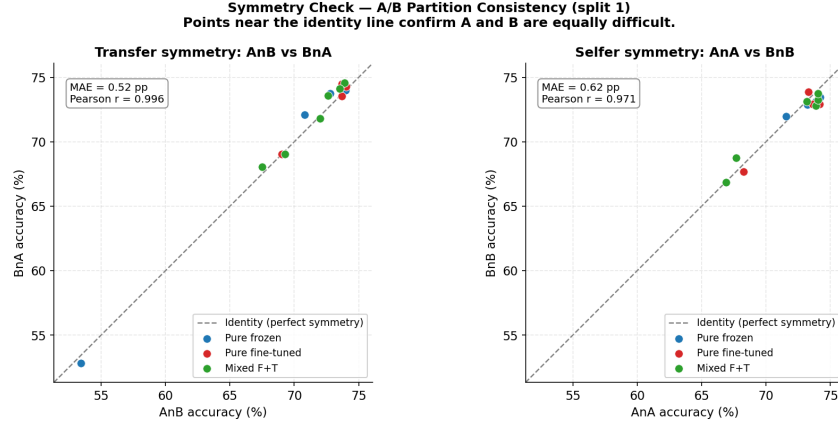


Figure 2: Split validity symmetry.

Yosinski-style replication. The pure frozen and pure fine-tuned curves reproduce the qualitative structure of Yosinski et al. (2014), but with different magnitudes. In the selfer frozen curve, co-adaptation fragility is present but small: accuracy drops from the scratch baseline of 73.7% to 71.8% at FFFS, then recovers to 73.5% at FFFF. This attenuation is consistent with WRN-28-10 residual connections reducing the brittleness caused by freezing intermediate representations. In cross-domain frozen transfer, the dominant effect is instead feature specialization. Performance remains near baseline through Conv1–Group2, but collapses at FFFF to 53.1%, compared with 73.5% for selfer FFFF. This indicates that the deepest WRN block is no longer reusable as a fixed representation when source and target class sets differ.

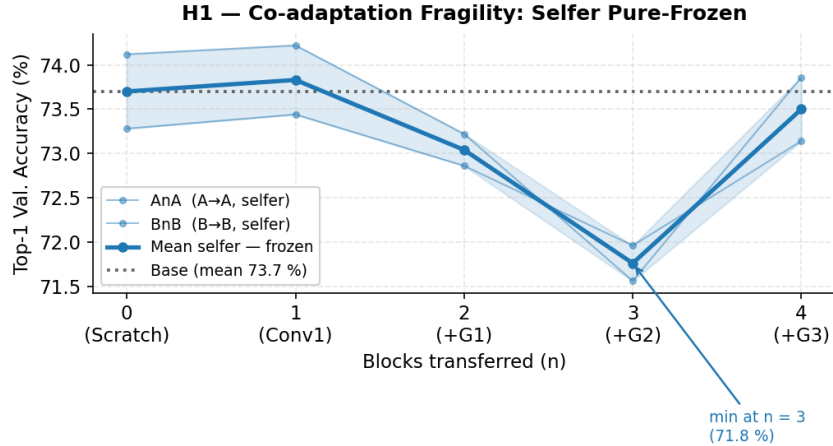


Figure 3: Co-adaptation fragility.

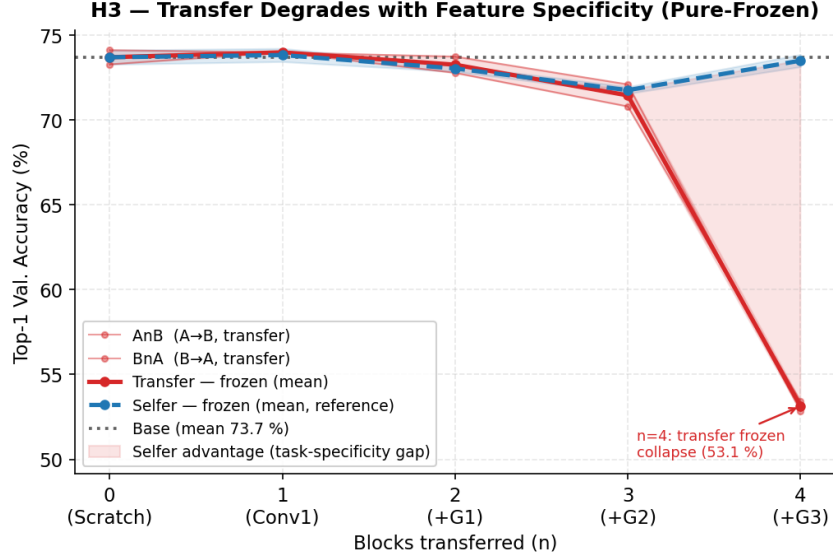


Figure 4: Transfer specificity collapse.

Fine-tuning recovery and the G2→G3 instability. Fine-tuning usually restores performance, but the ternary $F/T/S$ sweep reveals a failure mode that the original binary frozen/scratch design cannot show. Configurations where Group2 is fine-tuned while Group3 is scratch (TTTS, FTTS, and FFTS) all fall to roughly 67–69%. The effect appears in both selfer and transfer runs, so it is not primarily a domain-mismatch effect. The most plausible explanation is optimization-related: Group3 contains most of the WRN-28-10 parameters and is trained from scratch with a larger learning rate, while its input distribution changes because the upstream Group2 is being fine-tuned. This creates a moving-target boundary between T and S . Moving Group3 out of scratch consistently restores near-baseline performance.

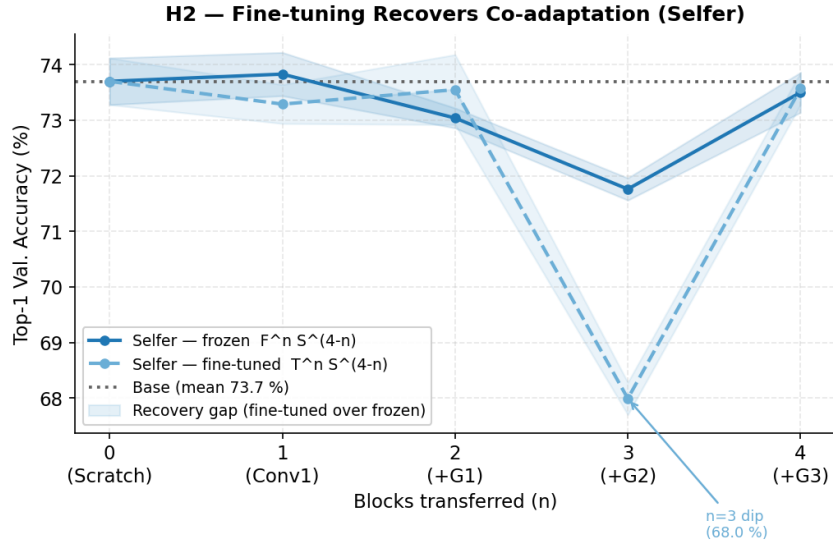


Figure 5: Fine-tuning recovery and TTTTS failure.

Corruption robustness. TinyImageNet-C-style corruption accuracy largely follows clean accuracy. Across the 56 source-dependent runs, clean and corrupted accuracies are strongly correlated (Pearson $r = 0.980$, Spearman $\rho = 0.910$). The synthetic corruption evaluation does not reveal a separate clean–robustness trade-off: the configurations that fail on clean validation are also the ones that fail under corruption. In particular, the G2→G3 instability and the FFFF transfer collapse remain the dominant failure modes. This should be interpreted

cautiously, because TinyImageNet-C is a synthetic perturbation benchmark and is not equivalent to the real location shift tested in Experiment 2.

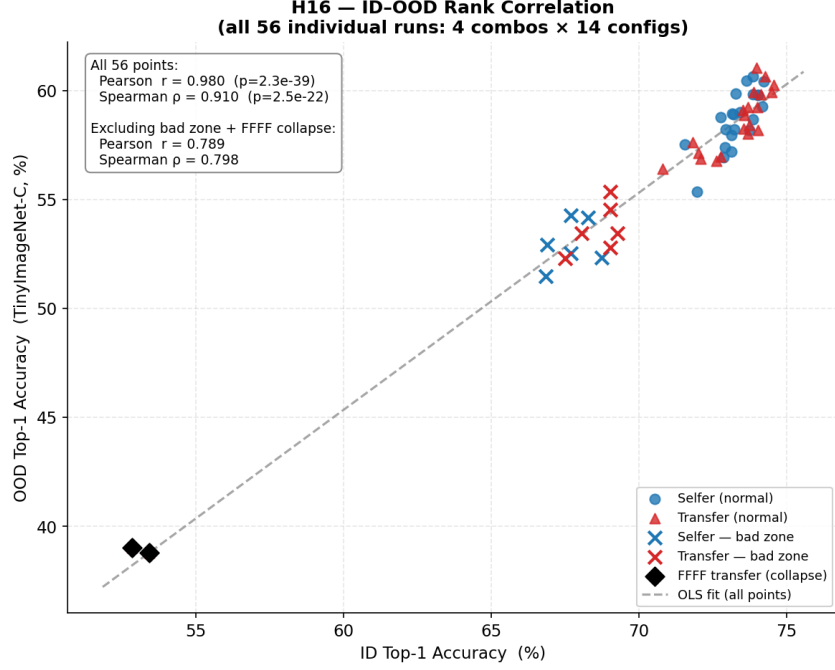


Figure 6: ID–OOD–C scatter.

D.2 Experiment 2: Real-World iWildCam Transfer

Experiment 2 tests the same $F/T/S$ principle in a practical transfer setting: ImageNet-pretrained ResNet50 adapted to iWildCam. Unlike Experiment 1, the source model is externally pretrained, the target domain is visually and semantically different, and the evaluation includes real location shift between seen and unseen camera-trap sites.

Frozen-feature transfer fails under large shift. The fully frozen configuration F5S1 is worse than training from scratch on raw accuracy: F5S1 obtains 42.8% ID and 22.8% OOD accuracy, while S6 obtains 56.5% ID and 30.3% OOD. This is the most direct practical warning from the experiment. A fixed ImageNet backbone plus a new classifier head is too rigid for camera-trap wildlife classification. This mirrors the FFFF collapse in Experiment 1, but under a larger and more realistic source–target gap.

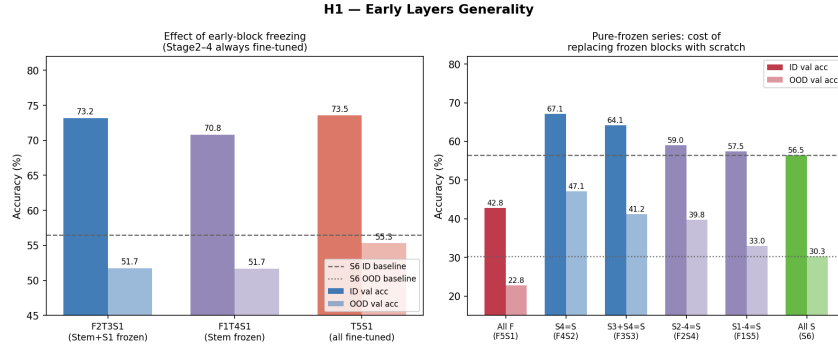


Figure 7: Top-layer specificity.

Stage4 is the key adaptation region. The largest gains are controlled by the final residual stage. Stage4 contains the majority of ResNet50 parameters and stores high-level ImageNet semantic features. When it is

frozen, the model is locked into source-task semantics; when it is scratch, it must relearn high-level representations from iWildCam data alone. Fine-tuning Stage4 is the only option that combines pretrained structure with target adaptation. This is visible in the transition from T4S2 to T5S1: replacing scratch Stage4 with fine-tuned Stage4 improves raw OOD accuracy by +11.2 percentage points. Full fine-tuning T5S1 is the best OOD configuration, reaching 55.3% raw OOD accuracy, 31.3% OOD balanced accuracy, and 29.5% OOD macro-F1.

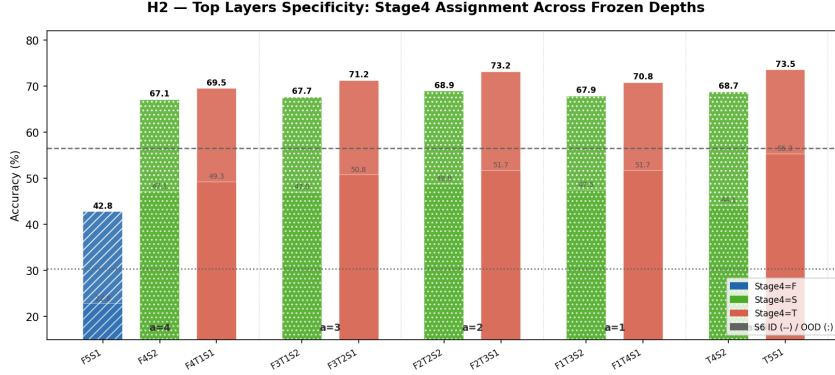


Figure 8: Top-layer specificity.

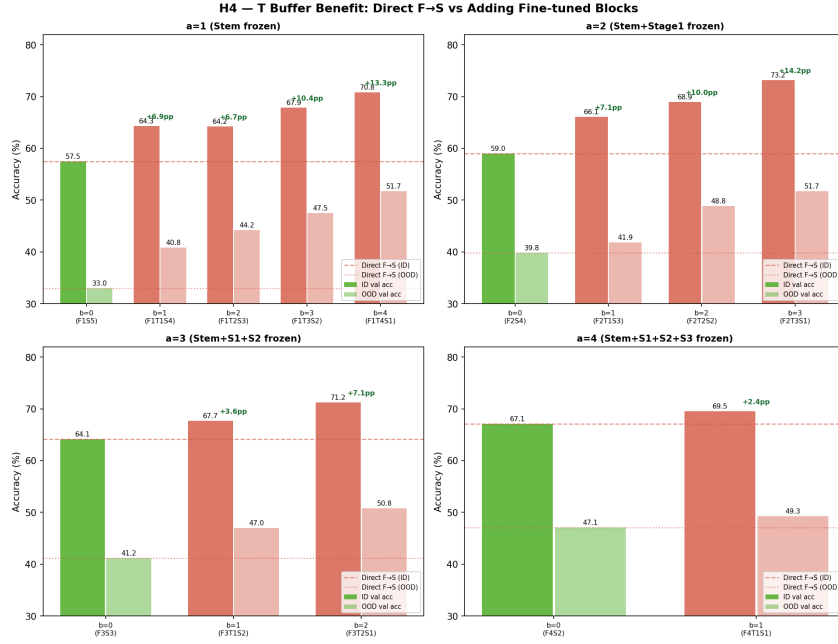


Figure 9: T-buffer bridging

Mixed partitions expose the efficiency frontier. Mixed *F/T/S* partitions are not a universal replacement for full fine-tuning, but they identify useful trade-offs. Fine-tuning only Stage4 (F4T1S1) reaches 49.3% raw OOD accuracy at about 76% of the full fine-tuning FLOPs. F3T2S1 and F2T3S1 move closer to full-finetuning performance, but require near-full compute. The frontier therefore has a sharp structure: most of the cheap gain comes from adapting Stage4, while the final accuracy gains require fine-tuning most of the backbone. This also explains why trainable-parameter percentage alone is not sufficient: Stage4 dominates parameters, but earlier stages still contribute substantially to FLOPs because they operate at larger spatial resolutions.

ID and OOD behavior are aligned. Across the 21 configurations, ID and OOD metrics are strongly aligned. The sweep does not show a regime where a configuration improves ID performance while sacrificing OOD generalization. Instead, weak configurations tend to fail both in-domain and under location shift. This

suggests that the $F/T/S$ partition mainly controls representation quality rather than inducing a clean overfitting-versus-robustness trade-off. Nevertheless, because iWildCam is class-imbalanced, the main comparisons should be read through macro-level metrics: OOD balanced accuracy, equivalent to macro recall, and OOD macro-F1.

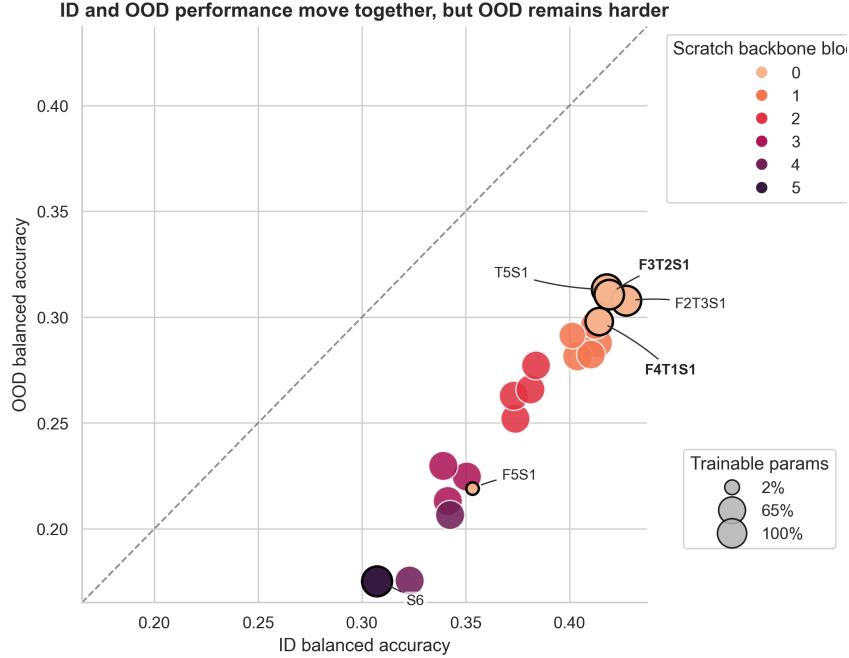


Figure 10: ID–OOD alignment.

D.3 Cross-Experiment Discussion

The two experiments support the same depth-wise view of transfer learning, but in different regimes. Experiment 1 is controlled: both source and target domains are constructed from TinyImageNet, and source networks are trained from scratch. This makes it possible to separate co-adaptation fragility from feature specialization. Experiment 2 is realistic: the source is ImageNet pretraining, the target is iWildCam, and the OOD split reflects unseen camera locations. This makes it possible to test whether the same partition logic remains useful under practical domain shift.

Increasing domain shift requires more adaptation. A consistent pattern emerges when moving from same-domain TinyImageNet selfer runs to cross-domain TinyImageNet transfer and then to ImageNet→iWildCam transfer. In the selfer case, large frozen prefixes remain safe and the best configuration freezes most of the backbone. In the cross-domain TinyImageNet case, freezing too deeply becomes risky and the best configuration fine-tunes most later blocks. In iWildCam, the best OOD configuration is full fine-tuning. Thus, as source–target mismatch increases, the preferred strategy moves from reuse toward adaptation.

Deep features are valuable but fragile. Both experiments show that the deepest stages are the most consequential. In TinyImageNet, the deepest WRN group causes the FFFF transfer collapse when frozen across domains and the G2→G3 instability when trained from scratch above a fine-tuned block. In iWildCam, Stage4 determines most of the performance gap between weak and strong configurations. The shared lesson is not that deep features should always be discarded; rather, they must be compatible with the target. If they are frozen from the wrong domain, they fail. If they are randomly initialized and placed above a drifting representation, they may be unstable. If they are pretrained and fine-tuned, they become the main source of transfer benefit.

Mixed partitions are diagnostic rather than universally optimal. The experiments do not show that mixed $F/T/S$ partitions always dominate full fine-tuning. On iWildCam, T5S1 is the best OOD configuration. The value of the partition sweep is instead diagnostic: it reveals which depth regions can be safely reused, which require adaptation, and which should not be relearned from scratch. This is why the same framework is useful even when the final recommendation differs across regimes.

Synthetic and real OOD behavior differ in scope. TinyImageNet-C-style corruption accuracy tracks clean accuracy almost perfectly, while iWildCam evaluates a broader real-world shift involving camera location, background, illumination, class imbalance, and animal appearance. The absence of a clean–corrupt trade-off in Experiment 1 should therefore not be interpreted as a general robustness result. It only shows that, for this controlled corruption set, the same configurations that learn good clean representations also perform well under synthetic perturbation. The iWildCam experiment is the more relevant test for real distribution shift.

Overall conclusion from the appendix. The combined evidence supports a precise but limited claim. Transfer learning is not solved by choosing a single global rule such as freezing early layers or fine-tuning everything. Instead, the best strategy depends on the interaction between architecture, initialization, and source–target mismatch. Early features are often reusable, but this is not enough under large shift. Deep semantic stages are powerful but require the right treatment: frozen when the task is the same, fine-tuned when the domain changes, and only cautiously relearned from scratch.

Why the TinyImageNet instability does not reappear. The $G2 \rightarrow G3$ instability from Experiment 1 does not have a direct analogue in iWildCam. The key difference is initialization. In TinyImageNet, the problematic block is randomly initialized from scratch and must learn while its input distribution drifts. In iWildCam, high-performing configurations initialize Stage4 from ImageNet weights before fine-tuning. This pretrained starting point stabilizes the deepest stage and makes the T region act as adaptation rather than as a moving boundary into a randomly initialized block. The comparison suggests that scratch suffixes are most dangerous when they are large, high-level, and trained immediately above a fine-tuned block without pretrained initialization.

E Limitations and Future Work

Our study is intended as a controlled diagnostic analysis rather than a state-of-the-art transfer-learning method. Several limitations should therefore be kept in mind.

Seeds and splits. The controlled TinyImageNet experiment uses a single semantically stratified split into two 100-class subsets, and the main experiments are not repeated over multiple random seeds. The two transfer directions $A \rightarrow B$ and $B \rightarrow A$ provide a useful symmetry check, but they do not replace a full estimate of split-to-split or seed-to-seed variance. Repeating the sweep across multiple class splits and initialization seeds would make the quantitative conclusions more robust.

Dataset scale. TinyImageNet is useful for controlled experimentation, but it is smaller and lower-resolution than ImageNet. Its two class subsets are also drawn from the same 200-class pool, so the source–target shift is limited compared with more realistic transfer scenarios. This likely reduces the size of the observed generalization boost and may make some transfer failures less severe than they would be on larger or more diverse datasets.

Architecture granularity. Both experiments use coarse block-level decompositions: four WRN-28-10 blocks and five ResNet50 blocks. This makes the $F/T/S$ sweep interpretable and computationally feasible, but it cannot identify finer transition points inside residual groups or bottleneck layers. Future work could repeat the analysis with finer block decompositions or layer-wise partitions.

Model family and adaptation methods. The experiments focus on convolutional backbones and do not evaluate modern pretrained architectures such as Vision Transformers, ConvNeXt-style models, or self-supervised backbones. They also do not compare against parameter-efficient adaptation methods such as adapters, LoRA-style updates, or visual prompt tuning. These methods are an important part of modern transfer learning, especially for large pretrained models, and would be a natural extension of the $F/T/S$ diagnostic framework.